

QUESTIONS 10 / 10 completed

Binary Search Tree ▾

BINARY SEARCH TREE

Write a C Program to create a Binary Search Tree by inserting the following elements and traverse it. 77, 33, 11, 99, 66, 55, 1, 2, 8, 88. Perform inorder, Preorder and Post Order.

PROGRAM EDITOR

C ▾

Run

Save

```
1 #include<stdio.h>
2 #include<stdlib.h>
3
4 struct node{
5     int data;
6     struct node *left,*right;
7 };
8
9 struct node* insert(struct node* r,int x){
10     if(r==NULL){
11         r=malloc(sizeof(struct node));
12         r->data=x;r->left=r->right=NULL;
13     }
14     else if(x<r->data) r->left=insert(r->left,x);
15     else r->right=insert(r->right,x);
16     return r;
17 }
18
19 void inorder(struct node*r){
20     if(r){inorder(r->left);printf("%d ",r->data);inorder(r->right);}
21 }
```

INPUT

77 33 11 99 66 55 1 2
8 88

QUESTIONS 10 / 10 completed

Inorder Traversal ▾

INORDER TRAVERSAL

"Create a Binary Search Tree and display its elements using Inorder Traversal - Create a node. Insert elements according to BST property. Perform Inorder Traversal. Display elements in sorted order."

PROGRAM EDITOR

C ▾

Run

Save

```
1 #include<stdio.h>
2 #include<stdlib.h>
3
4 struct node{
5     int data;
6     struct node *left,*right;
7 };
8
9 struct node* insert(struct node*r,int x){
10     if(r==NULL){
11         r=malloc(sizeof(struct node));
12         r->data=x;r->left=r->right=NULL;
13     }
14     else if(x<r->data) r->left=insert(r->left,x);
15     else r->right=insert(r->right,x);
16     return r;
17 }
18
19 void inorder(struct node*r){
20     if(r){
21         inorder(r->left);
```

INPUT

```
10
77 33 11 99 66 55 1 2
8 88
```

QUESTIONS 10 / 10 completed

BST

BST

Write a C Program to search the element 100 is available in the constructed BST using following element and traverse it. 77, 33, 11, 99, 66, 55, 1, 2, 8, 88.

PROGRAM EDITOR

C

Run

Save

```
1 #include<stdio.h>
2 #include<stdlib.h>
3
4 struct node{int data;struct node*l,*r;};
5
6 struct node*insert(struct node*t,int x){
7     if(!t){
8         t=malloc(sizeof(struct node));
9         t->data=x;t->l=t->r=NULL;
10    }else if(x<t->data)t->l=insert(t->l,x);
11    else t->r=insert(t->r,x);
12    return t;
13 }
14
15 int search(struct node*t,int x){
16     if(!t)return 0;
17     if(t->data==x)return 1;
18     if(x<t->data)return search(t->l,x);
19     return search(t->r,x);
20 }
```

INPUT

```
77 33 11 99 66 55 1 2
8 88
```

QUESTIONS 10 / 10 completed

Binary Search-Traverse ▾

BINARY SEARCH-TRAVERSE

Construct a Binary Search tree using the following characters(ASCII) " C, O, M, P, U, T, E, R " and Traverse it using C Programming.

PROGRAM EDITOR

C ▾

Run

Save

```
1 #include<stdio.h>
2 #include<stdlib.h>
3
4 struct node{char data;struct node*l,*r;};
5
6 struct node*insert(struct node*t,char x){
7     if(!t){
8         t=malloc(sizeof(struct node));
9         t->data=x;t->l=t->r=NULL;
10    }else if(x<t->data)t->l=insert(t->l,x);
11    else t->r=insert(t->r,x);
12    return t;
13 }
14
15 void inorder(struct node*t){
16     if(t){inorder(t->l);printf("%c ",t->data);inorder(t->r);}
17 }
18
19 int main(){
20     struct node*t=NULL;
21     char a[]={'C','O','M','P','U','T','E','R'};
```

INPUT

COMPUTER

QUESTIONS 10 / 10 completed

Binary Search Tree- ▾

BINARY SEARCH TREE-2

Delete an element '25' from the constructed Binary Search Tree having the elements of 66, 33, 88, 22, 25, 44, 99, 11, 55. Implement using C Programming.

PROGRAM EDITOR

C ▾

Run

Save

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 struct node {
5     int data;
6     struct node *left, *right;
7 };
8
9 struct node* insert(struct node *r, int x) {
10     if (r == NULL) {
11         r = malloc(sizeof(struct node));
12         r->data = x;
13         r->left = r->right = NULL;
14     }
15     else if (x < r->data)
16         r->left = insert(r->left, x);
17     else
18         r->right = insert(r->right, x);
19     return r;
20 }
21
```

INPUT

```
9
66 33 88 22 25 44 99
11 55
25
```

QUESTIONS 10 / 10 completed

BST-Minimum & N

BST-MINIMUM & MAXIMUM

Write a C program to find Minimum and Maximum Element in a Binary Search Tree and find the difference between them.

PROGRAM EDITOR

C

Run

Save

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 struct node {
5     int data;
6     struct node *left, *right;
7 };
8
9 struct node* insert(struct node *r, int x) {
10     if (r == NULL) {
11         r = malloc(sizeof(struct node));
12         r->data = x;
13         r->left = r->right = NULL;
14     } else if (x < r->data)
15         r->left = insert(r->left, x);
16     else
17         r->right = insert(r->right, x);
18     return r;
19 }
20
21 int main() {
```

INPUT

```
9
66 33 88 22 25 44 99
11 55
```

QUESTIONS 10 / 10 completed

BST-String

BST-STRING

Construct a Binary Search tree using the following String " Car, Owl , Man, Puppy, Uday, Tata, Elite, Relay " and Traverse it using C Programming.

PROGRAM EDITOR

C



Run

Save

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <string.h>
4
5 struct node {
6     char data[20];
7     struct node *left, *right;
8 };
9
10 struct node* insert(struct node *root, char s[]) {
11     if (root == NULL) {
12         root = malloc(sizeof(struct node));
13         strcpy(root->data, s);
14         root->left = root->right = NULL;
15     } else if (strcmp(s, root->data) < 0)
16         root->left = insert(root->left, s);
17     else
18         root->right = insert(root->right, s);
19
20     return root;
21 }
```

INPUT

8
Car
Owl
Man
Puppy
Uday
Tata
Elite
Relay

QUESTIONS 10 / 10 completed

BST-2

BST-2

Print all the odd elements present in the constructed Binary Search tree using following elements. 22, 88, 66, 55, 33, 11, 99, 100, 77, 44. Implement using C Programming.

PROGRAM EDITOR

C



Run

Save

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 struct node {
5     int data;
6     struct node *l, *r;
7 };
8
9 struct node* insert(struct node *p, int x) {
10     if (!p) {
11         p = malloc(sizeof(struct node));
12         p->data=x; p->l=p->r=NULL;
13     } else if(x<p->data) p->l=insert(p->l,x);
14     else p->r=insert(p->r,x);
15     return p;
16 }
17
18 void odd(struct node *p) {
19     if(p) {
20         odd(p->l);
21         if(p->data%2) printf("%d ", p->data);
```

INPUT

```
22 88 66 55 33 11 99
100 77 44
```

QUESTIONS 10 / 10 completed

Constructed Binary ▾

CONSTRUCTED BINARY SEARCH TREE

Print the count of prime numbers present in the constructed Binary Search tree using following elements. 22, 88, 66, 55, 33, 11, 99, 100, 77, 44. Implement using C Programming.

PROGRAM EDITOR

C ▾

Run

Save

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 struct node {
5     int data;
6     struct node *l,*r;
7 };
8
9 struct node* insert(struct node *p,int x) {
10     if(!p) {
11         p=malloc(sizeof(struct node));
12         p->data=x; p->l=p->r=NULL;
13     } else if(x<p->data) p->l=insert(p->l,x);
14     else p->r=insert(p->r,x);
15     return p;
16 }
17
18 int prime(int n) {
19     if(n<2) return 0;
20     for(int i=2;i*i<=n;i++)
21         if(n%i==0) return 0;
```

INPUT

```
22 88 66 55 33 11 99
100 77 44
```